ASSIGNMENT 2

Instructions for Students

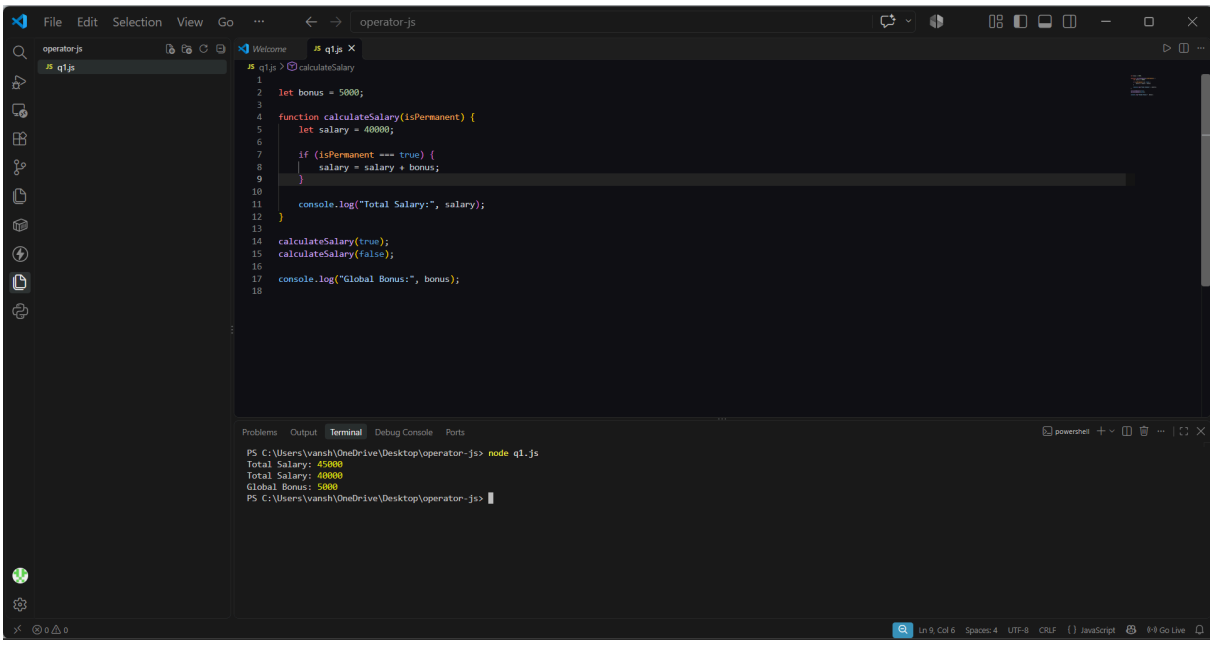
- Create **separate `.js` files** for each question.
- Test all code in the **browser console** or **Node.js** environment.
- Use **meaningful variable names** and **comment** each logic section.
- Use **no external libraries or frameworks**.
- Maintain proper **indentation** and **output clarity**.

Q1. Scope Conflict Resolver

A company's payroll script has variables defined in multiple scopes.
You must correct the logic using proper variable scoping.

Requirements:

- Declare a global variable `bonus = 5000`.
- Inside a function `calculateSalary()`, declare a local variable `salary = 40000`.
- Add the bonus only if a local variable `isPermanent` (boolean) is true. ● Print total salary inside the function.
- Demonstrate how changing `isPermanent` affects the global variable scope.

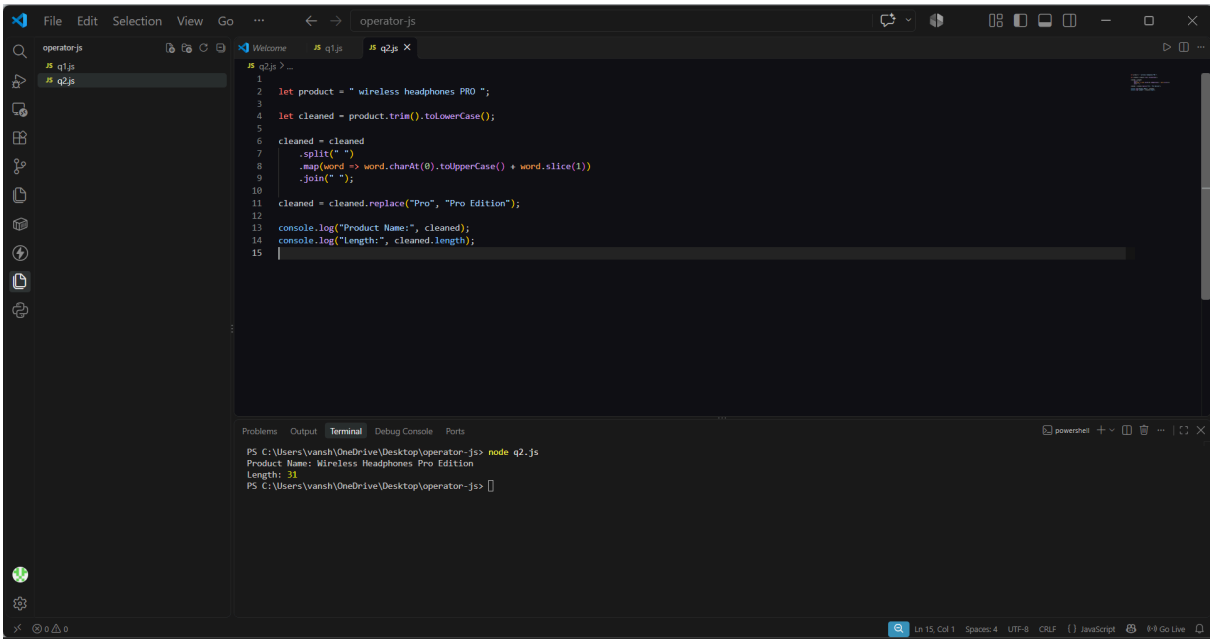


Q2. String Manipulation Report

An e-commerce site wants a script that formats product titles properly before display.

Requirements:

- Store a product name like `"  wireless headphones PRO  "`.
- Trim extra spaces, convert to lowercase, then capitalize the first letter of each word.
- Replace `"pro"` with `"Pro Edition"`.
- Display the cleaned title and its length.
- Use methods: `trim()`, `split()`, `map()`, `join()`, `replace()`, and `length`.

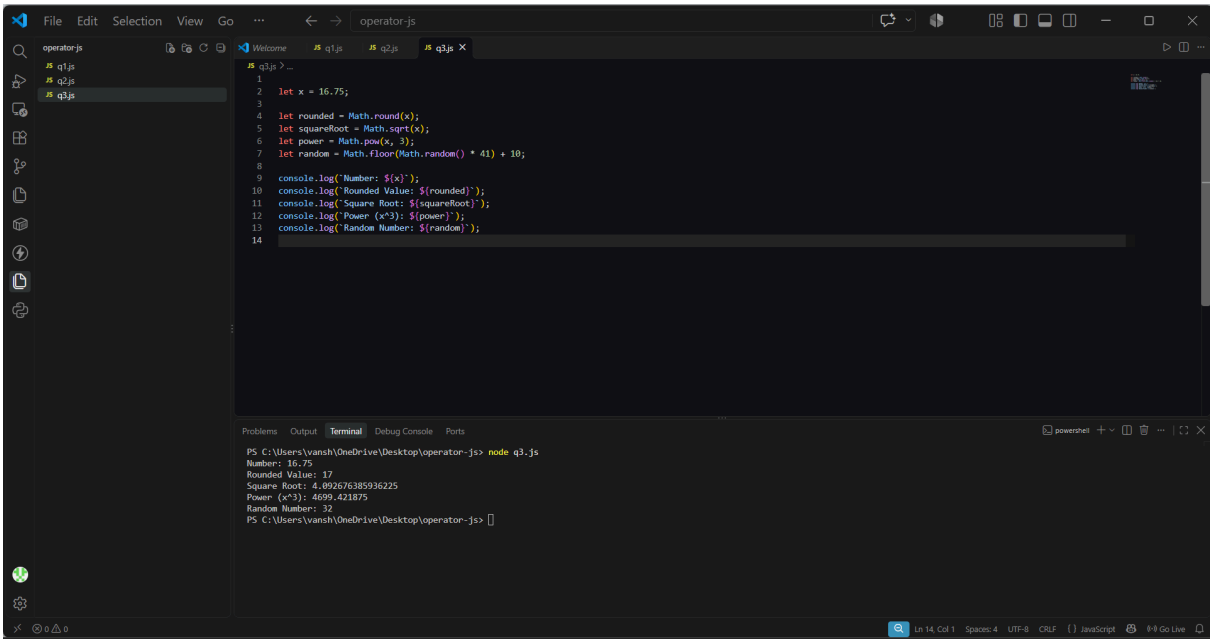


Q3. Math Utility Dashboard

You’re creating a math dashboard for a scientific calculator.

Requirements:

- Take a number `x = 16.75`.
- Display:
    - Rounded value (`Math.round`)
    - Square root (`Math.sqrt`)
    - Power (`Math.pow(x, 3)`)
    - Random number between 10–50 (`Math.floor(Math.random() * 41) + 10`)
- Use template literals to show a formatted result summary.

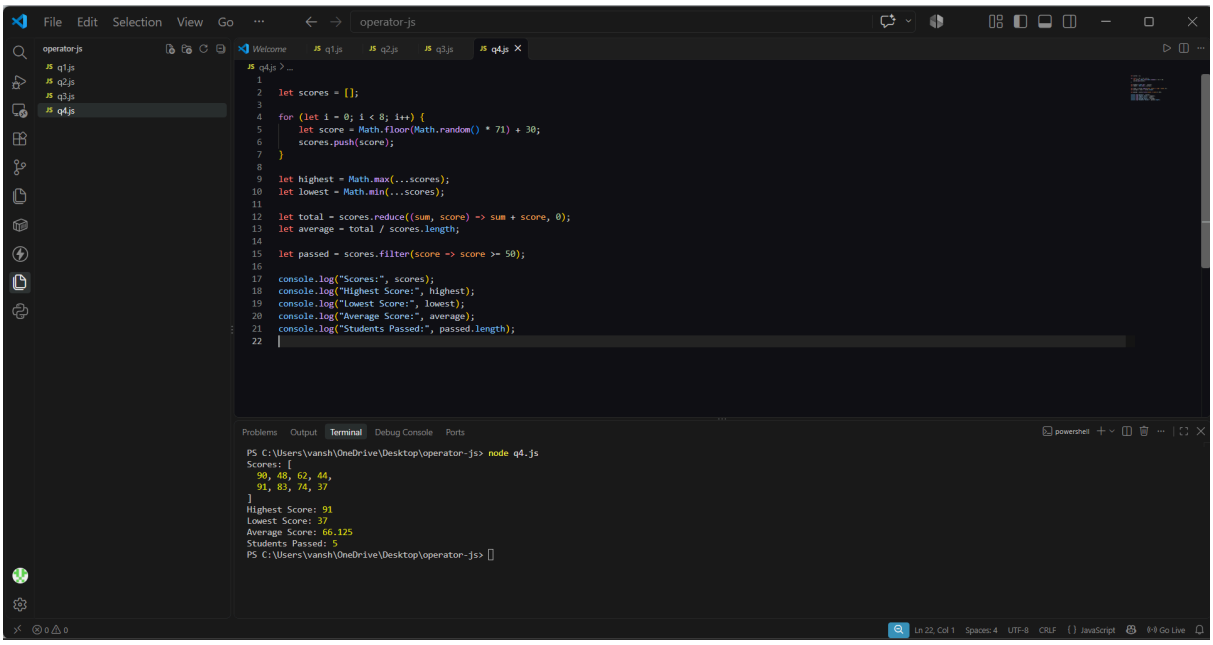


Q4. Array Performance Analyzer

You need to analyze student performance using an array of scores.

Requirements:

- Create an array of 8 random scores between 30 and 100.
- Calculate:
    - Highest and lowest score (`Math.max(...arr)`, `Math.min(...arr)`)
    - Average score
    - Number of students who passed (≥ 50)
- Use array methods like `map()`, `filter()`, and `reduce()`.
- Display a full summary in a formatted output.

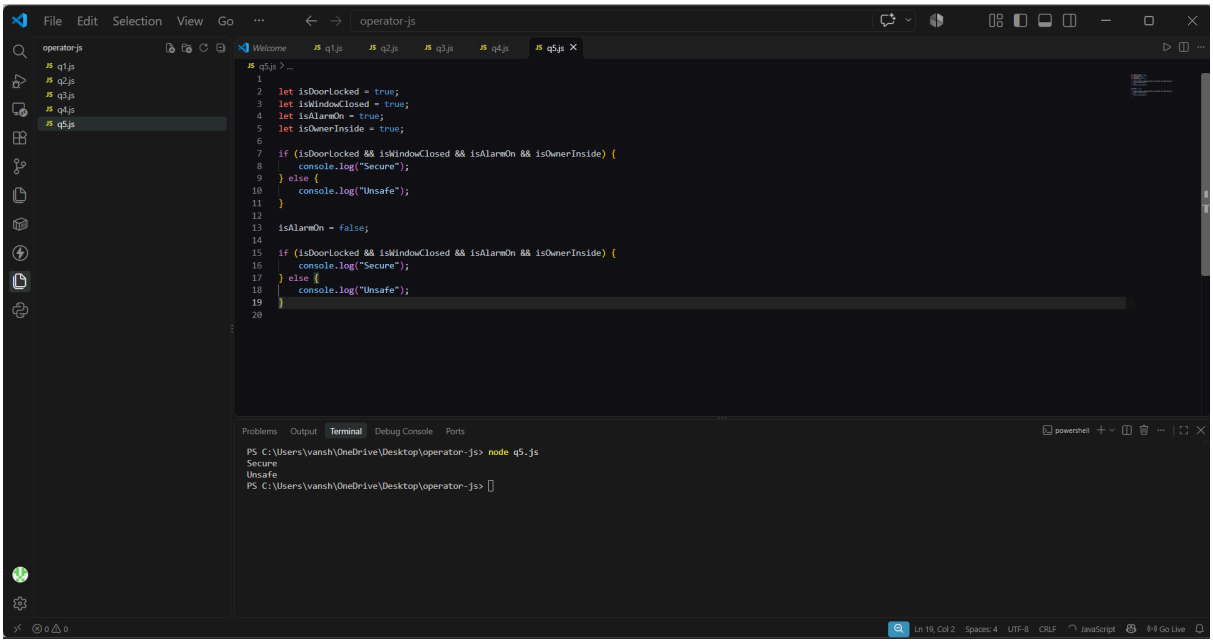


Q5. Boolean Logic Access System

A smart home system checks multiple security conditions.

Requirements:

- Variables: `isDoorLocked`, `isWindowClosed`, `isAlarmOn`, `isOwnerInside`.
- Access is **granted** only if:
    - Alarm is on
    - Door and window are closed
    - Owner is inside
- Use Boolean logic (`&&`, `||`, `!`) to print “Secure” or “Unsafe”.
- Change values and test multiple outcomes.

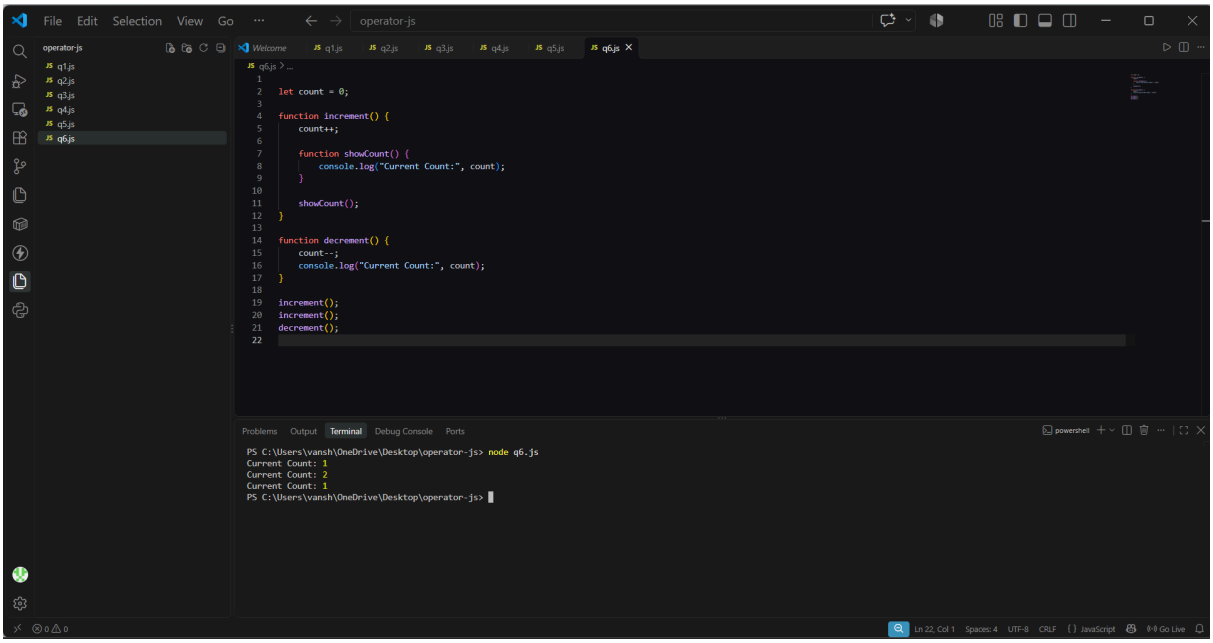


Q6. Event-Based Counter Simulation

Create a simple counter that increases or decreases based on simulated click events.

Requirements:

- Use variables `count = 0`.
- Create two functions: `increment()` and `decrement()`.
- Each function modifies `count` and logs the current value.
- Simulate “click” by calling these functions manually in code.
- Use a nested function to show **function scope** of count updates.



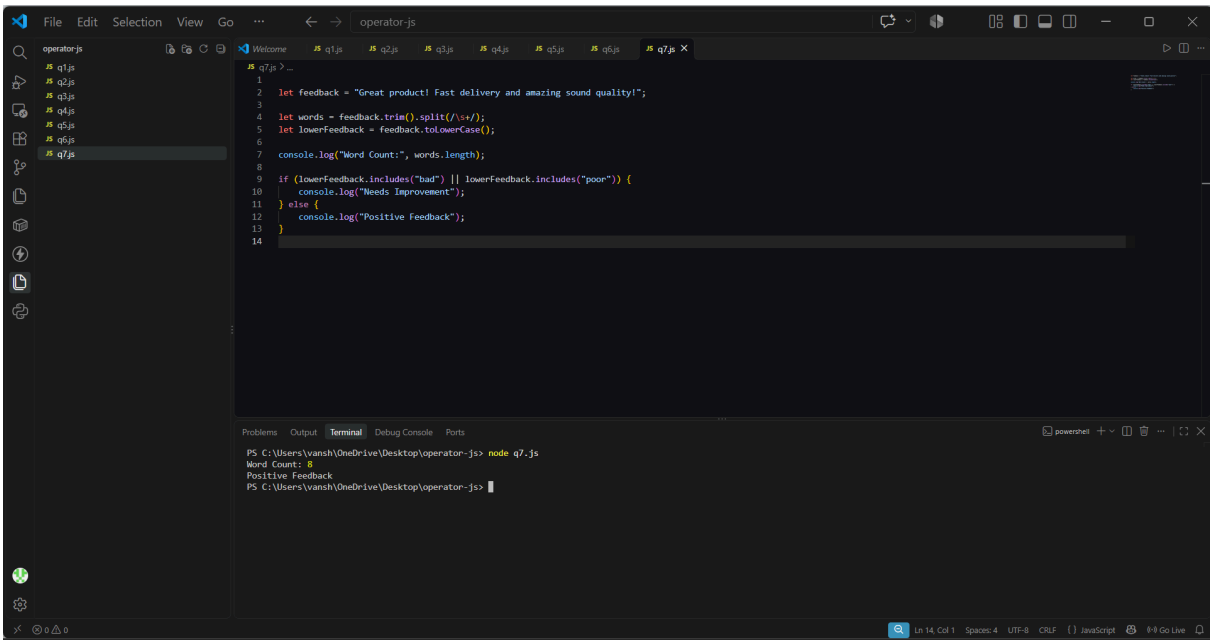
Q7. Customer Feedback Processor

A review system must analyze comments for positivity and length.

Requirements:

- Store a feedback string like `"Great product! Fast delivery and amazing sound quality!"`.
- Use string methods to:
    - Count words
    - Check if the feedback includes `"bad"` or `"poor"`
    - If not → print `"Positive Feedback"`
    - Else → `"Needs Improvement"`

- Use `includes()`, `split()`, and conditional statements.



Q8. Dynamic Discount Evaluator

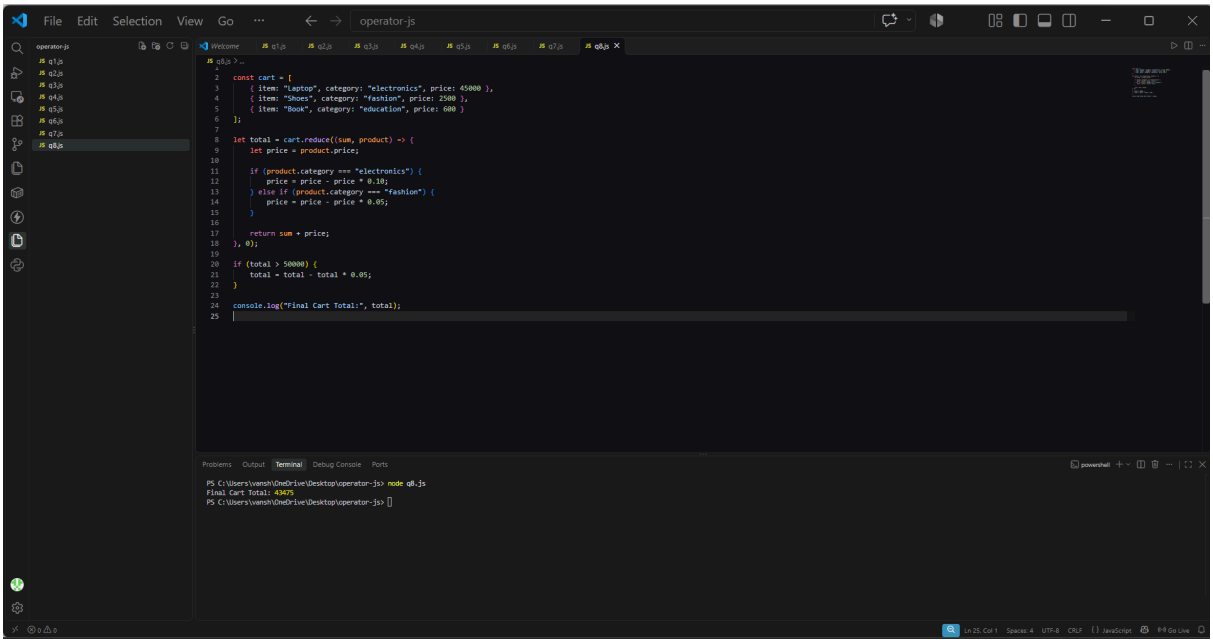
A store gives discounts based on product categories and cart totals.

Requirements:

Use an array of objects:

```
const cart = [
  { item: "Laptop", category: "electronics", price: 45000 },
{ item: "Shoes", category: "fashion", price: 2500 }, { item:
"Book", category: "education", price: 600 } ];
```

-
- If category = “electronics” → 10% discount
- If category = “fashion” → 5% discount
- If total cart value > 50000 → apply extra 5% overall discount.
- Use loops, conditionals, and `reduce()` to calculate the final total.

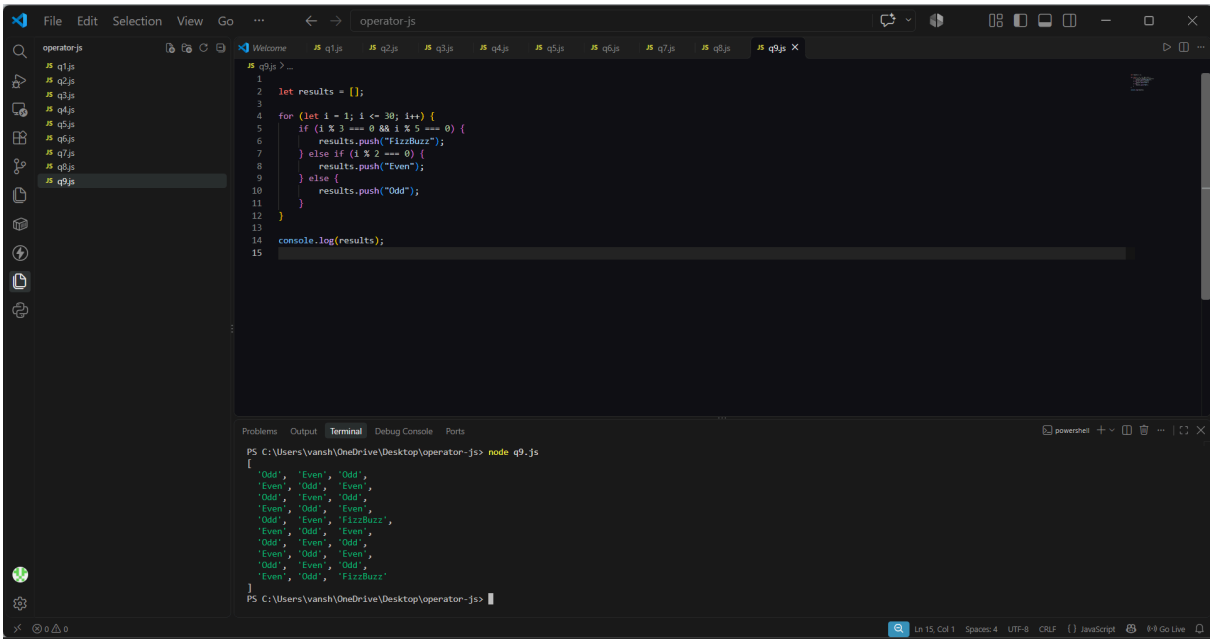


Q9. Odd–Even Number Analyzer

You are designing a tool that classifies numbers in a range based on multiple conditions.

Requirements:

- Accept numbers 1–30 in an array using a loop.
- For each number:
    - If divisible by 3 and 5 → print “FizzBuzz”
    - Else if even → print “Even”
    - Else → print “Odd”
- Store results in a new array and display it.
- Use %, `push()`, and conditionals.



Q10. Departmental Employee Evaluator

You must evaluate employees from multiple departments based on performance points.

Requirements:

Create a nested array for departments:

```
const departments = [
  ["HR", 72],
  ["Finance", 88],
  ["Tech", 95],
  ["Support", 63]
];
```

-
- Use loops and conditionals to print:
    - “Excellent” if score ≥ 90
    - “Good” if 75–89

- “Average” if 60–74
- “Needs Improvement” otherwise

- Use comparison operators and nested `if-else` conditions.\

